Machine Coding: Throttled Search

- **Problem:** Build a live product search (`dummyjson.com/products/search?q=`) that updates results **while** the user is still typing, but calls the API **at most once per second**, and always searches the final text when typing stops. Add a switch to compare Throttle / Debounce / None, and a request log that makes the rate visible. Handle loading, empty, error + Retry, too-short queries, stale responses and repeated queries.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Interval, leading/trailing decided
Use cases + architecture + state	7 min	Table, hook split, timeline sketch
Plain JS `throttle` + self-check	10 min	Throttle proven without React
`useThrottle` hook + search hook	13 min	Live results, ≤ 1 request/s
States, abort, cache, mode switch, request log	12 min	Correct + visible
Test manually (log / Network tab) + wrap-up	13 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Draw the timeline as you talk: *"Keys arrive every 150 ms. With a 1 s throttle I send the first value immediately (leading), ignore the rest of the window, and at the end of the window send the **latest** value (trailing), so the last thing the user typed is never lost."* Then show the request log while typing.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Live search: results refresh during typing, not only after a pause.
- At most one API request per 1000 ms while the input keeps changing.
- **Leading:** the first change is sent immediately. **Trailing:** the final text is always searched at the end of the window.
- Ignore empty / too-short input (< 2 characters after trim) with a hint.
- States: "Searching...", results, "No products found", error + Retry.
- A mode switch (Throttle / Debounce / None) and a request log with timestamps to compare them.
- Old responses must never overwrite newer ones; repeated queries come from a cache.

Non-functional requirements

Concern	What it means here
Performance	Typing for 10 s sends ~10 requests instead of ~60; the input itself is never throttled.
Scalability	A predictable maximum request rate per user (1/s), which is easy to size the server for.
Error handling	`response.ok` + shape checks, `AbortError` ignored, Retry, clean-up of timers on unmount.
Accessibility	Labelled input, `role="search"`, `fieldset` / `legend` for the mode radios, `aria-live` status, `role="alert"` error, `aria-busy` on stale results.
Reusability	`throttle(fn, wait, options)` for plain JS; `useThrottle(value, interval)` for React values; the search hook is shared with the debounce version.
Extensibility	Change the interval or leading/trailing options in one place; plug in another API in `api/products.js`.

Clarifying questions to ask the interviewer

1. Why throttle instead of debounce here: do they want results *while* typing (throttle) or only after a pause (debounce)?
2. What interval: 500 ms, 1 s? Is there an API rate limit we must stay under?
3. Leading, trailing, or both? Must the final text always be searched?
4. Minimum characters? Should Enter search immediately?
5. Keep old results while the next request loads, or clear them?
6. Is this also used for other events (scroll, resize, button clicks)? Then a plain JS throttle helper is needed too.

Assumptions we will build with

- Throttle 1000 ms with **leading + trailing** (first and last value always sent).
- Minimum 2 characters after trim.

- Old results stay visible, faded, while the next search is pending.
- An in-memory cache per query; stale requests are aborted.
- Debounce (400 ms) and None modes exist only to compare the behaviour.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type "smartphones" steadily (1 key / 150 ms ≈ 1.6 s)	Requests at ~0 s and ~1 s (latest text each time), the last one = "smartphones"; results update during typing
Same typing in **None** mode	One request per keystroke (≥ 2 chars)
Same typing in **Debounce** mode	One request, after typing stops
Type 1 character	"Type at least 2 characters."; no request
Paste "laptop" in one go	Sent immediately (leading)
Stop typing mid-window	The latest text is sent at the end of the window (trailing)
Older request still in flight when a newer term is sent	Older one aborted; never overwrites
API returns products	"N results for 'x'" + list
API returns `[]`	`No products found for "x".`
API fails	`Search failed (HTTP 404).` + Retry
Search a term already searched	Served from the cache; no request logged
Click ×	Input cleared; results cleared; pending timers dropped
Switch the mode	The next changes follow the new rate limit; the request log shows the difference

3. Design the architecture

Module	Responsibility
`App`	Owns `query` + `mode`; picks the rate-limited value; derives UI flags; renders everything
`utils/throttle.js`	Plain JS `throttle(fn, wait, { leading, trailing })` with `cancel()`
`hooks/useThrottle.js`	`useThrottle(value, interval)`: a value updated at most once per interval, leading + trailing
`hooks/useDebounce.js`	`useDebounce(value, delay)` for the comparison mode
`hooks/useProductSearch.js`	Fetch for a term: abort stale, cache, retry, request log
`api/products.js`	`searchProducts(query, { signal })`: URL encoding, `ok` + shape checks
`ResultItem` / `RequestLog`	One product row / the timeline of real requests

```
App  (query, mode)
 │  throttled = useThrottle(query, 1000)
 │  debounced = useDebounce(query, 400)
 │  limited   = { throttle, debounce, none: query }[mode]
 │  searchTerm = limited.trim() (≥ 2 chars) or ""
 │  { status, results, error, log, retry } = useProductSearch(searchTerm)
 ├── mode radios (fieldset)
 ├── input + clear
 ├── status (aria-live) / error + Retry
 ├── <ul> results → ResultItem
 └── RequestLog ← log
utils/throttle.js  ← plain JS version (event handlers, scroll, interview question)
```

Start simple. First version: the existing "button + plain throttle" idea, then a search that fetches on every keystroke (show the flood), then `useThrottle`, then abort/cache/log.

4. Design the state / data model

State	Where	Purpose
`query`	App	Raw input value, every keystroke
`mode`	App	`throttle | debounce | none` (demo)
`throttled`	useThrottle	Latest value published by the throttle
`lastRun` (ref)	useThrottle	Time of the last publish; a ref because it must not cause renders
`debounced`	useDebounce	Value after a pause (comparison)
`status`, `results`, `error`	useProductSearch	Request state (`idle | loading | success | error`)
`log`	useProductSearch	`[{ id, term, at }]` for the request timeline
`attempt`	useProductSearch	Retry counter
`cache` (ref)	useProductSearch	`Map<term, products>`

```
const [query, setQuery] = useState("");
const [mode, setMode] = useState("throttle");

const throttled = useThrottle(query, THROTTLE_MS); // hooks always run (rules of hooks)
const debounced = useDebounce(query, DEBOUNCE_MS);
const limited = { throttle: throttled, debounce: debounced, none: query }[mode];

const term = limited.trim();
const searchTerm = term.length >= MIN_LENGTH ? term : "";
const { status, results, error, log, retry } = useProductSearch(searchTerm);

const typed = query.trim();
```

```
const isPending = typed !== term && typed.length >= MIN_LENGTH; // limiter still holding the latest value
const isBusy = isPending || status === "loading";
```

Senior note

- **Derived, not stored:** `limited`, `searchTerm`, `isTooShort`, `isPending`, `isBusy` and the request count (`log.length`). The only user-owned state is `query` and `mode`.
- **Refs vs state:** `lastRun` and `cache` are bookkeeping that must survive renders but must not *cause* them, so they are refs.
- **Hooks can't be conditional:** both `useThrottle` and `useDebounce` always run; `mode` only selects which value is used. (In a real app you'd keep just one.)
- **Ownership:** `App` owns what the user typed; the hooks own timing and networking.

5. Design the data structures and contracts

API response (trimmed with `select=`):

```
{ "products": [ { "id": 121, "title": "iPhone 5s", "category": "smartphones", "price": 199.99, "thumbnail": "https://…" } ],
  "total": 23, "skip": 0, "limit": 10 }
```

Request log entry (stable `id` from `crypto.randomUUID()` for React keys; `at` in ms):

```
{ "id": "5c1e…", "term": "smartph", "at": 1793560000123 }
```

Contracts

```
throttle(fn, wait, { leading = true, trailing = true }) → throttled(...args), throttled.cancel()
useThrottle(value, interval)  → value updated ≤ 1× per interval (leading + trailing)
useDebounce(value, delay)     → value after `delay` ms of no change
useProductSearch(term)        → { status, results, error, log, clearLog, retry }   // term "" → idle
searchProducts(query, { signal }) → Promise<Product[]>  // GET /products/search?q={encoded}&limit=10
```

6. Trace the data flow

User types "smartphones" at one key every 150 ms, Throttle mode:

```
t=0.00s  "s"           → useThrottle: window open → publish "s" (leading) → too short → no request
t=0.15…0.90s "sm"…"smartph" → inside the window → one trailing timer, each new value replaces the pending one
t=1.00s  timer fires   → publish "smartph" → useProductSearch: cache miss → abort old, fetch ?q=smartph
                         → log "+0.0s smartph" → results update while the user is still typing
t=1.05…1.50s "smartpho"…"smartphones" → new window → trailing timer again
```

```
t=2.00s  timer fires   → publish "smartphones" → previous request aborted if still running → fetch
                         → log "+1.0s smartphones" → final results; "11 chars → 2 requests"
```

7. Important technical challenges

Challenge 1: writing a correct throttle (leading + trailing)

- *Problem:* The first version only has `if (now - lastCall >= delay)`. That's leading-only: the user types "smartphones", the call for "s" runs, everything else in the window is **dropped**, and the final text is never searched.
- *Solution:* Remember the latest arguments and schedule one trailing call for the end of the window.

```
const remaining = wait - (now - lastRun);
if (remaining <= 0) { clearTimeout(timer); timer = null; run(args); }
else if (trailing) {
  lastArgs = args;                                  // keep only the latest call
  if (!timer) timer = setTimeout(() => { timer = null; run(lastArgs); }, remaining);
}
```

Challenge 2: throttling a value in React

- *Problem:* `useCallback(throttle(fetchApi, 10000), [])` needs an eslint-disable, freezes `fetchApi` with the state from the first render (stale closure), and its timer never gets cleaned up on unmount.
- *Solution:* Throttle the **value** with a hook. A ref keeps `lastRun`; the effect either publishes now or schedules the trailing publish, and the cleanup drops the pending one when a newer value arrives.

```
useEffect(() => {
  if (Object.is(value, throttled)) return;
  const remaining = interval - (Date.now() - lastRun.current);
  const publish = () => { lastRun.current = Date.now(); setThrottled(value); };
  if (remaining <= 0) { publish(); return; }
  const timer = setTimeout(publish, remaining);
  return () => clearTimeout(timer);
}, [value, interval, throttled]);
```

Challenge 3: race conditions get *more* likely with throttle

- *Problem:* Throttle sends requests *while* typing, so several can overlap. "smartph" (slow, 2 s) can arrive after "smartphones" (fast) and replace the final results with old ones.
- *Solution:* One `AbortController` per request, aborted in the effect cleanup; `AbortError` ignored. A cache avoids re-sending the same term.

```
const controller = new AbortController();
searchProducts(key, { signal: controller.signal }).then(…).catch((err) => { if (err.name !==
"AbortError") … });
return () => controller.abort();
```

Challenge 4: proving the behaviour (throttle vs debounce)

- *Problem:* "It feels fine" is not proof, and interviewers often ask "what exactly is the difference?".
- *Solution:* A request log with relative timestamps, and a mode switch. The same typing gives: **None** → one request per key; **Debounce** → one request after the pause; **Throttle** → one request per second during typing, plus the final one.

```
setLog((prev) => [...prev, { id: crypto.randomUUID(), term: key, at: Date.now() }]);
// RequestLog: +0.0s q="smartph", +1.0s q="smartphones"
```

8. Implement in the correct order

#	Step	Working result
1	Plain `throttle` (leading only) on a button click	Basic idea working
2	Add trailing + `cancel`; node self-check (leading, trailing, last value, cancel)	Correct throttle, proven
3	Controlled input + fetch on every change (no limit)	Works, request flood visible
4	`useThrottle(query, 1000)` drives the search term	≤ 1 request/s, final text searched
5	Minimum length + trim + hint	No useless requests
6	`status` enum, `ok` + shape checks, error + Retry	Async states
7	Move fetching into `useProductSearch`	Clean `App`
8	`AbortController` cleanup	No stale overwrites
9	Cache `Map` in a ref	Repeated terms are free
10	Request log + mode switch (Throttle / Debounce / None)	Behaviour visible
11	`isPending` / `isBusy`, faded stale results, clear button	Smooth UX
12	`aria-live`, `role="alert"`, `fieldset`, labels	Accessible

9. Cover all states and edge cases

State / edge case	Handling
Initial (idle)	Empty input, "No API requests yet."
Typing, throttle window open	"Searching 'x'...", old results faded
Loading	`aria-busy`, faded results
Success	"N results for 'x'" + list
Empty result	`No products found for "x".`
Error	HTTP status message + Retry
1 character / only spaces	Hint / idle; no request
First change	Sent at once (leading), if ≥ 2 chars

State / edge case	Handling
Typing stops mid-window	Latest text sent at the window end (trailing)
Value goes back to the last published one before the timer fires	Pending timer dropped; nothing sent
Overlapping requests	Older ones aborted
Same term again	Cache hit; not logged
Special characters	`encodeURIComponent`
Unmount / clear while a timer or request is pending	Cleanups clear the timer and abort the request
Mode switch mid-typing	The next value follows the new mode; hooks stay mounted

10. Final structure

```
_06_throttle_search/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css                ← global reset
    ├── App.jsx                  ← query + mode, derived flags, UI
    ├── App.css
    ├── api/
    │   └── products.js          ← searchProducts (encode, ok + shape checks)
    ├── utils/
    │   └── throttle.js          ← plain JS throttle (leading/trailing/cancel)
    ├── hooks/
    │   ├── useThrottle.js       ← throttle a value
    │   ├── useDebounce.js       ← debounce a value (comparison)
    │   └── useProductSearch.js  ← fetch + abort + cache + retry + log
    └── components/
        ├── ResultItem.jsx
        └── RequestLog.jsx
```

11. Full working code

`src/utils/throttle.js`

```
// Plain JS throttle: runs `fn` at most once per `wait` ms.
// leading: run on the first call. trailing: run once more at the end with the LAST arguments.
export function throttle(fn, wait, { leading = true, trailing = true } = {}) {
  let lastRun = 0;
  let timer = null;
  let lastArgs = null;
```

```
  const run = (args) => {
    lastRun = Date.now();
    fn(...args);
  };

  const throttled = (...args) => {
    const now = Date.now();
    if (!lastRun && !leading) lastRun = now; // without leading, the first window starts now
    const remaining = wait - (now - lastRun);

    if (remaining <= 0) {
      clearTimeout(timer);
      timer = null;
      run(args);
    } else if (trailing) {
      lastArgs = args; // remember only the latest call
      if (!timer) {
        timer = setTimeout(() => {
          timer = null;
          run(lastArgs);
        }, remaining);
      }
    }
  };

  throttled.cancel = () => {
    clearTimeout(timer);
    timer = null;
    lastRun = 0;
  };
  return throttled;
}
```

src/hooks/useThrottle.js

```
import { useEffect, useRef, useState } from "react";

// Returns `value`, updated at most once per `interval` ms while it keeps changing.
// Leading: the first change shows at once. Trailing: the last value always arrives.
export function useThrottle(value, interval) {
  const [throttled, setThrottled] = useState(value);
  const lastRun = useRef(0);

  useEffect(() => {
    if (Object.is(value, throttled)) return; // nothing new to publish
```

```
    const remaining = interval - (Date.now() - lastRun.current);
    const publish = () => {
      lastRun.current = Date.now();
      setThrottled(value);
    };

    if (remaining <= 0) {
      publish();
      return;
    }
    const timer = setTimeout(publish, remaining); // trailing call at the end of the window
    return () => clearTimeout(timer); // a newer value replaces this pending one
  }, [value, interval, throttled]);

  return throttled;
}
```

src/hooks/useDebounce.js

```
import { useEffect, useState } from "react";

// Returns `value` only after it has stopped changing for `delay` ms (used here for comparison).
export function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debounced;
}
```

src/api/products.js

```
const API_URL = "https://dummyjson.com/products/search";

// shortcut: ?fail in the page URL makes the first request fail, to demo the error state.
let failNext = new URLSearchParams(window.location.search).has("fail");

// GET /products/search?q=phone&limit=10 -> { products: [{ id, title, category, price, thumbnail }],
total }
export async function searchProducts(query, { signal } = {}) {
  const url = `${failNext ? API_URL + "-broken" : API_URL}?
q=${encodeURIComponent(query)}&limit=10&select=title,category,price,thumbnail`;
  failNext = false;
```

```
  const response = await fetch(url, { signal });
  if (!response.ok) throw new Error(`Search failed (HTTP ${response.status}).`);
  const data = await response.json();
  if (!Array.isArray(data.products)) throw new Error("Unexpected response from the server.");
  return data.products;
}
```

src/hooks/useProductSearch.js

```
import { useEffect, useRef, useState } from "react";
import { searchProducts } from "../api/products";

const IDLE = { status: "idle", results: [], error: "" };

// Fetches results for an (already rate-limited) term. Aborts stale requests, caches by term,
// and logs every real request so the UI can show how often the API was hit.
export function useProductSearch(term) {
  const [state, setState] = useState(IDLE); // status: idle | loading | success | error
  const [log, setLog] = useState([]); // [{ id, term, at }]
  const [attempt, setAttempt] = useState(0);
  const cache = useRef(new Map());

  useEffect(() => {
    const key = term.trim().toLowerCase();
    if (!key) {
      setState(IDLE);
      return;
    }
    if (cache.current.has(key)) {
      setState({ status: "success", results: cache.current.get(key), error: "" });
      return;
    }

    const controller = new AbortController();
    setState((prev) => ({ ...prev, status: "loading", error: "" })); // keep old results while loading
    setLog((prev) => [...prev, { id: crypto.randomUUID(), term: key, at: Date.now() }]);
    searchProducts(key, { signal: controller.signal })
      .then((results) => {
        cache.current.set(key, results);
        setState({ status: "success", results, error: "" });
      })
      .catch((err) => {
        if (err.name === "AbortError") return; // a newer term replaced this request
        setState({ status: "error", results: [], error: err.message });
      });

    return () => controller.abort();
```

```
  }, [term, attempt]);

  return {
    ...state,
    log,
    clearLog: () => setLog([]),
    retry: () => setAttempt((a) => a + 1),
  };
}
```

src/App.jsx

```
import { useState } from "react";
import "./App.css";
import { useThrottle } from "./hooks/useThrottle";
import { useDebounce } from "./hooks/useDebounce";
import { useProductSearch } from "./hooks/useProductSearch";
import ResultItem from "./components/ResultItem";
import RequestLog from "./components/RequestLog";

const THROTTLE_MS = 1000;
const DEBOUNCE_MS = 400;
const MIN_LENGTH = 2;
const MODES = [
  { value: "throttle", label: `Throttle (${THROTTLE_MS} ms)` },
  { value: "debounce", label: `Debounce (${DEBOUNCE_MS} ms)` },
  { value: "none", label: "None" },
];

function App() {
  const [query, setQuery] = useState(""); // updates on every keystroke
  const [mode, setMode] = useState("throttle");

  // Hooks always run (rules of hooks); the mode only picks which value drives the search.
  const throttled = useThrottle(query, THROTTLE_MS);
  const debounced = useDebounce(query, DEBOUNCE_MS);
  const limited = { throttle: throttled, debounce: debounced, none: query }[mode];

  const term = limited.trim();
  const searchTerm = term.length >= MIN_LENGTH ? term : "";
  const { status, results, error, log, clearLog, retry } = useProductSearch(searchTerm);

  const typed = query.trim();
  const isTooShort = typed.length > 0 && typed.length < MIN_LENGTH;
  const isPending = typed !== term && typed.length >= MIN_LENGTH; // limiter still holding the latest
value
  const isBusy = isPending || status === "loading";
```

```
  return (
    <main className="container">
      <h1>Throttled Product Search</h1>

      <fieldset className="modes">
        <legend>Rate limit</legend>
        {MODES.map((m) => (
          <label key={m.value}>
            <input type="radio" name="mode" value={m.value} checked={mode === m.value} onChange={() =>
setMode(m.value)} />
            {m.label}
          </label>
        ))}
      </fieldset>

      <div className="search-box" role="search">
        <label htmlFor="search" className="sr-only">
          Search products
        </label>
        <input
          id="search"
          type="search"
          placeholder="Type fast, e.g. smartphone"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          autoComplete="off"
          autoFocus
        />
        {query && (
          <button type="button" onClick={() => setQuery("")} aria-label="Clear search">
            ×
          </button>
        )}
      </div>

      <div aria-live="polite" className="status">
        {isTooShort && <p>Type at least {MIN_LENGTH} characters.</p>}
        {!isTooShort && isBusy && <p>Searching "{typed}"...</p>}
        {!isBusy && status === "success" && results.length === 0 && <p>No products found for "{term}".
</p>}
        {!isBusy && status === "success" && results.length > 0 && <p>{results.length} results for "
{term}"</p>}
      </div>

      {status === "error" && !isPending && (
        <div role="alert" className="error">
```

```
          <p>{error}</p>
          <button type="button" onClick={retry}>
            Retry
          </button>
        </div>
      )}

      {results.length > 0 && searchTerm && (
        <ul className={`results ${isBusy ? "stale" : ""}`} aria-busy={isBusy}>
          {results.map((p) => (
            <ResultItem key={p.id} product={p} />
          ))}
        </ul>
      )}

      <RequestLog log={log} onClear={clearLog} />
    </main>
  );
}

export default App;
```

src/components/ResultItem.jsx

```
const ResultItem = ({ product }) => (
  <li className="result">
    <img src={product.thumbnail} alt="" width="44" height="44" loading="lazy" />
    <div>
      <p className="result-title">{product.title}</p>
      <p className="result-meta">
        {product.category} · ${product.price}
      </p>
    </div>
  </li>
);

export default ResultItem;
```

src/components/RequestLog.jsx

```
// Shows each real API request with the time since the first one, so the rate limit is visible.
const RequestLog = ({ log, onClear }) => {
  if (log.length === 0) return <p className="log-empty">No API requests yet.</p>;
  const start = log[0].at;

  return (
```

```
    <section className="log" aria-label="API request log">
      <div className="log-header">
        <strong>API requests: {log.length}</strong>
        <button type="button" className="link" onClick={onClear}>
          Clear log
        </button>
      </div>
      <ol>
        {log.slice(-8).map((entry) => (
          <li key={entry.id}>
            <code>+{((entry.at - start) / 1000).toFixed(1)}s</code> q="{entry.term}"
          </li>
        ))}
      </ol>
    </section>
  );
};

export default RequestLog;
```

src/App.css

```
body { background: #f3f4f6; font-family: system-ui, sans-serif; }
.container { max-width: 620px; margin: 32px auto; padding: 20px; background: white; border-radius: 10px;
}
h1 { text-align: center; margin-bottom: 16px; font-size: 24px; }

.modes { display: flex; flex-wrap: wrap; gap: 12px; border: 1px solid #e5e7eb; border-radius: 8px;
padding: 8px 12px; margin-bottom: 12px; font-size: 14px; }
.modes legend { padding: 0 4px; font-weight: 600; }
.modes label { display: flex; gap: 4px; align-items: center; cursor: pointer; }

.search-box { display: flex; gap: 8px; }
.search-box input { flex: 1; padding: 10px 12px; font-size: 16px; border: 1px solid #ccc; border-radius:
8px; }
.search-box input:focus { outline: 2px solid #4f46e5; border-color: transparent; }
button { padding: 8px 14px; border: none; border-radius: 8px; background: #4f46e5; color: white; cursor:
pointer; font-size: 16px; }
button.link { background: none; color: #4f46e5; padding: 0; font-size: 13px; }

.status { min-height: 24px; margin: 12px 0 8px; color: #4b5563; }
.error { color: #dc2626; display: flex; align-items: center; justify-content: space-between; gap: 8px;
margin-bottom: 8px; }

.results { list-style: none; border: 1px solid #e5e7eb; border-radius: 8px; transition: opacity 0.2s; }
.results.stale { opacity: 0.5; }
.result { display: flex; gap: 12px; align-items: center; padding: 8px 10px; border-bottom: 1px solid
```

```
#f3f4f6; }
.result:last-child { border-bottom: none; }
.result img { border-radius: 6px; object-fit: cover; background: #f3f4f6; }
.result-title { font-weight: 600; }
.result-meta { font-size: 13px; color: #6b7280; text-transform: capitalize; }

.log { margin-top: 16px; padding: 10px 12px; background: #f9fafb; border-radius: 8px; font-size: 13px; }
.log-header { display: flex; justify-content: space-between; margin-bottom: 6px; }
.log ol { padding-left: 20px; color: #374151; }
.log code { color: #4f46e5; margin-right: 6px; }
.log-empty { margin-top: 16px; font-size: 13px; color: #9ca3af; }

.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-
space: nowrap; }
```

`src/index.css`

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: leading and trailing calls (first and last value never lost), minimum length, overlapping requests (abort), repeated terms (cache), URL encoding, HTTP and shape errors with Retry, timers and requests cleaned up on unmount/clear, mode switching, visible proof via the request log.

Trade-offs to mention to the interviewer

- *Throttle vs debounce for search:* throttle gives live feedback while typing but sends more requests; debounce sends the fewest, but shows nothing until the user pauses. Many products combine them

(throttle for live preview, or debounce with `maxWait`).

- *Interval choice:* shorter = more responsive but more load; 300 ms-1 s is typical. Match the API rate limit.
- *Leading + trailing:* the best UX (fast first response, correct final result), but the last call can come up to `interval` ms after the user stops.
- *`Date.now()` vs `performance.now()`:* `Date.now()` can jump if the system clock changes; `performance.now()` is monotonic. Fine either way for a UI throttle.
- *Value hook vs throttled function:* the value hook is idiomatic React with no stale closures; the plain function is right for DOM events (scroll, resize, mousemove), wrapped in `useMemo` with a cleanup.
- *Throttling with timers vs `requestAnimationFrame`:* for visual updates (scroll position, drag), rAF throttling (≈16 ms, synced to paint) is better than `setTimeout`.

What to add with more time

- Tests: `throttle` and `useThrottle` with fake timers (advance 999 ms → nothing; 1 ms → trailing call with the last value); a mocked `fetch` for abort and cache.
- `maxWait` for debounce (a hybrid), and `flush()` on Enter.
- Cache with TTL / LRU, or React Query.
- A scroll-position / infinite-scroll example using the same `throttle` + `requestAnimationFrame`.
- An autocomplete dropdown with keyboard navigation.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Throttle vs debounce: explain with a timeline.

Keys every 150 ms for 1.6 s. **Debounce (400 ms):** nothing while typing, one call 400 ms after the last key. **Throttle (1 s):** a call at 0 s, one at 1 s, and a trailing call at 2 s with the final text. Debounce = "wait for quiet"; throttle = "at most once per interval, even while busy".

Q2. When would you choose throttle over debounce for search?

When users expect live results while typing (instant search, typeahead suggestions), and the API can handle a steady rate. If requests are expensive or rate-limited hard, use debounce.

Q3. What other events need throttling?

Scroll (infinite scroll, sticky headers, scroll spy), resize, mousemove / drag, window `pointermove` for tooltips, button spam (rate-limit "send" or "refresh"), analytics events, and live position updates in collaborative apps.

Q4. What does "leading" and "trailing" mean? Which do you choose?

Leading: run on the first call of a window. Trailing: run once at the end of the window with the latest arguments. For search, both: instant first response and the final text is never lost. For "prevent double click", leading only.

State design

Q5. Why is `lastRun` a ref and not state?

It's bookkeeping read inside the effect; updating it must not cause a re-render. State would trigger extra renders and could loop.

Q6. Why does `useThrottle` have `throttled` in its dependency array?

The effect compares the new value with the last published one (`Object.is(value, throttled)`). After publishing, the effect runs again, sees they're equal, and stops. It also satisfies the exhaustive-deps rule without an eslint-disable.

Q7. Why do both `useThrottle` and `useDebounce` run even in "None" mode?

Hooks must be called in the same order on every render; they can't be inside an `if` . Both run cheaply and `mode` only picks which value to use. In a real app you'd keep just one.

Hard parts

Q8. Write a basic throttle in plain JavaScript.

```
function throttle(fn, wait) {
  let last = 0;
  return function (...args) {
    const now = Date.now();
    if (now - last >= wait) { last = now; fn.apply(this, args); }
  };
}
```

Then say it's leading-only, and extend it with a trailing timer (as in `utils/throttle.js`).

Q9. What's wrong with `useCallback(throttle(fetchApi, 10000), [])` ?

`throttle(...)` is still *called* on every render (the result is just ignored after the first), `fetchApi` is frozen with the first render's state (stale closure), it needs an eslint-disable, and its timer isn't cleaned up on unmount. Use a value hook, or `useMemo` + a ref to the latest callback + `cancel()` in cleanup.

Q10. Can a throttled search show wrong results? How do you prevent it?

Yes: throttle sends several requests while typing, and an older, slower one can arrive last. Abort the previous request when a new term is sent (`AbortController`), or ignore responses whose term is not the current one.

Q11. How would you throttle with `requestAnimationFrame` instead of a timer?

```
function rafThrottle(fn) {
  let frame = null, lastArgs;
  return (...args) => {
    lastArgs = args;
    if (frame) return;
    frame = requestAnimationFrame(() => { frame = null; fn(...lastArgs); });
```

```
  };
}
```

At most once per frame (~60/s), synced with painting: ideal for scroll and drag visuals.

Q12. How is debounce with `maxWait` related to throttle?

Debounce with `maxWait = N` waits for a pause, but forces a call at least every N ms during continuous input. With `maxWait === wait`, it behaves like a throttle. This is how lodash implements `throttle`.

Implementation steps

Q13. How do you test throttle?

With fake timers: call 10 times in 100 ms → 1 call (leading); advance to the end of the window → 1 more call with the *last* arguments (trailing); `cancel()` → the pending call never runs. Here a node script checks this with small real delays.

Q14. How did you prove the throttle works in the UI?

The request log with relative timestamps: typing "smartphones" steadily gives `+0.0s "smartph"`, `+1.0s "smartphones"`. In None mode the same typing gives one request per key; in Debounce mode only one.

Q15. What happens on unmount while a trailing call is pending?

The effect cleanup clears the timer, and the search hook aborts any in-flight request. A plain `throttle` used in React needs `cancel()` in the cleanup, or it may call `setState` after unmount.

Performance and scale

Q16. 10,000 users type at the same time. Is a client-side throttle enough?

No. It limits *each* client to ~1 request/s, which is still up to 10,000 requests/s. You also need server-side rate limiting (per IP / user), caching of popular queries (CDN / Redis), and maybe a larger interval or debounce.

Q17. The API is rate-limited to 5 requests/second per user. How do you set the throttle?

An interval ≥ 200 ms, with a margin (e.g. 300-500 ms), plus handling HTTP 429: back off (wait the `Retry-After` time) and show a gentle message instead of an error.

Q18. Scroll handler runs 100 times a second and the page janks. What do you do?

Throttle with `requestAnimationFrame`, mark the listener `{ passive: true }`, avoid layout reads/writes in the handler (layout thrashing), or replace it with `IntersectionObserver` (no scroll handler at all) for infinite scroll / lazy loading.

Trade-offs and "why not X?"

Q19. Why not lodash `throttle`?

In production, yes: it's battle-tested (leading/trailing/cancel/flush, `maxWait` logic). In an interview they want to see closures, timers and edge cases, so write it, then mention lodash.

Q20. Why not `useDeferredValue` or `useTransition`?

They keep the UI responsive by delaying expensive *rendering*, but they don't limit how often an effect fires or how many API requests are sent. Use them for heavy local filtering; use throttle/debounce for network calls.

Q21. Why does the throttle use `Date.now()` and not just a boolean "waiting" flag with `setTimeout`?

A flag version (`if (waiting) return; waiting = true; setTimeout(() => waiting = false, wait)`) is simpler but loses the trailing call and can drift. Tracking `lastRun` lets us compute the exact `remaining` time and schedule a correct trailing call.